

简单数据类型

字符串基本操作与输入输出

第3讲 · Python 基础

开始学习 →

学习目标

学完本节后，你能够：

- 调用常见字符串方法：strip / split / join / 大小写 / 替换 / 查找
- 用 input() 获取输入，并用 int() / float() 正确转换类型
- 用 f 字符串 / format 实现格式化输出
- 理解 strip / split / 格式化在 AI 文本处理中的作用

回顾

1. Python中简单数据类型有哪些？

(int、float、bool、str)

2. 如何查看一个变量的类型？

type()

3. '123' 和 123 一样吗？

(不一样，前者是字符串，后者是整数)

4. 字符串定义与分片：

```
s = "Python"
print(s[0], s[-1])    # P n
print(s[0:3])         # Pyt
```



今天学字符串的操作方法——注意：方法与分片是两种语法形态。

开场思考点

Python 字符串能否为空?

试一试（交互模式）：

```
s = ""          # 空字符串
print(len(s))   # 0
print(bool(s))  # False
print(bool(" ")) # True — 注意：空格不是空!
```



一句话：空字符串 `""` 长度为 0；但含空格 `" "` 不是空

字符串方法：调用方式

- 字符串方法用 `s.方法名( ... )` 调用，就像"对字符串做动作"
- 与分片（`s[ .. ]`）的区别：方法是**调用函数**，分片是**取片段**

```
s = " Python "
```

<code>print(s.strip())</code>	# 去两端空白: "Python"
<code>print(s.lower())</code>	# 全小写
<code>print(s.upper())</code>	# 全大写



重点方法 ①：strip 清洗

`strip()` 去掉字符串两端的空白（空格、换行、Tab）——AI 文本清洗第一步：

```
raw = "  欢迎学习Python  \n"  
print(repr(raw.strip()))      # '欢迎学习Python'
```



`lstrip()` / `rstrip()`：只去左 / 右端空白。

★ **AI 赋能：**自然语言处理中，爬取/读取的文本常带多余空白，`strip()` 先清洗。

重点方法 ②：split 分句 / 分词

split(分隔符) 把字符串按分隔符拆成列表：

```
s = "今天上课,讲了,字符串"
print(s.split(","))          # ['今天上课', '讲了', '字符串']
words = "I love Python".split()
print(words)                  # ['I', 'love', 'Python']
```



★ AI 赋能：对话文本分句——按句号/换行把长篇对话拆成一句句再处理。

重点方法 ③：join 拼接

`join()` 把列表合成字符串——与 `split` 相反（一个“拆”、一个“合”）：

```
words = ['I', 'love', 'Python']  
print(" ".join(words))      # I love Python  
print("-".join(words))      # I-love-Python
```



！ 记法： `"分隔符".join(列表)`，分隔符写在前。

其他常用方法

```
s = "Hello, Python"
print(s.replace("Hello", "Hi"))    # Hi, Python
print(s.find("Python"))            # 7 (找不到返回 -1)
print(s.count("o"))                # 2
print(len(s), "Python" in s)       # 13 True
print(s.startswith("He"))          # True
print(s.endswith("on"))            # True
```



- `len(s)`：长度； `in`：成员判断——注意是函数 / 运算符，不是方法。

常见错误：方法与分片混淆

错误	原因	正确做法
<code>s.upper</code> 漏括号	忘记方法调用要加 <code>()</code>	<code>s.upper()</code>
<code>s.upp</code> 拼写错	大小写/拼写	方法都用小写： <code>s.upper()</code>
<code>s.upper ()</code> 多空格	多余空格	<code>s.upper()</code>
把 <code>split</code> 当合并	方向记反	<code>split</code> 拆、 <code>join</code> 合

输入输出：input()

input() 从键盘读入一行文本，永远返回字符串：

```
name = input("请输入你的名字：")    # 提示词只显示，不进入变量
print("你好，" + name)
```



！ 关键：即使你输入的是数字，input() 返回的也是字符串！

难点！： 输入类型的转换

忘记转换 → 报错 (TypeError)

```
n = input("请输入数字: ")    # 假设输入 3
print(n + 5)                 # ✗ TypeError: can only concatenate str (not "int") to str
print(n * 2)                 # ⚠ 得到 "33", 是字符串重复, 不是 6!
```



正确做法 —— 先转换再运算

```
n = int(input("请输入数字: "))    # 不要再输入相同变量已占用, 新变量
print(type(input("请输入数字: "))) # <class 'str'>    ← 看清楚它的类型
x = int(input("a = "))            # 转换
y = float(input("b = "))          # 小数用 float
print(x + y)                     # 正常数值运算
```



难点❗：转换易错点小结

常见错误	后果	正确做法
用 <code>int()</code> 转小数	报 <code>ValueError</code>	小数用 <code>float()</code>
直接对 <code>input()</code> 结果做运算	<code>TypeError</code>	先 <code>int()</code> / <code>float()</code>
用户输入了非数字	<code>ValueError</code>	先了解，本讲不展开异常处理
转换后再重复用原字符串	混淆	转换后赋值给数值变量

格式化输出：f 字符串（推荐）

在字符串前加 `f`，用 `{}` 填变量，最直观：

```
name, score = "小明", 87.5
print(f"姓名: {name}, 成绩: {score}")      # 姓名: 小明, 成绩: 87.5
print(f"成绩保留一位: {score:.1f}")      # 87.5
```



`format()` / `%` 也可以：

```
print("姓名: {}, 成绩: {}".format(name, score))
print("成绩 %.1f" % score)
```



★ **AI 赋能：** 格式化生成 AI 模型提示词模板——把用户输入填充进预设模板。

print() 的参数

```
print(1, 2, 3)           # 1 2 3 (默认空格分隔)
print(1, 2, 3, sep="-")  # 1-2-3 (改分隔符)
print("a", end=" ")      # 结尾不换行, 改空格
print("b")               # 接着同一行: a b
```



AI 赋能场景：文本清洗与提示词模板

```
# ① strip 清洗 NLP 文本噪声
text = " 我正在学习中文自然语言处理  \n"
clean = text.strip()

# ② split 对话文本分句
dialog = "你好\n我想学Python\n应该怎么开始"
for line in dialog.split("\n"):
    print("句: " + line)

# ③ 格式化生成 AI 提示词模板
topic = "字符串"
prompt = f"请你用通俗的语言，为初学者讲解 Python 的{topic}。"
print(prompt)
```



实验任务（上机）

-  **基础任务（必做）**：字符串方法清理文本 + 输入两个数求和/平均
-  **进阶任务（选做）**：规范化清洗英文（split+join+首字母大写） + f 字符串自我介绍
-  **挑战任务（选做）**：对话文本分句统计 或 AI 提示词模板生成器

提交要求：源文件 + 运行结果截图

思考点

Python 字符串能否为空？

1. `""` 与 `" "` 有什么区别？长度分别是多少？
2. `bool("")`、`bool("abc")` 分别是 True 还是 False？
3. 在清洗文本时，为什么 `" "`（空格）不能当作空处理？

小结 + 预告

- 本课要点：字符串方法（重点 strip/split/join）、输入转换（难点）、f 字符串格式化
- 下次课：**组合数据类型**（列表、元组）
- 预习：Python 的组合数据类型；完成 HW02 作业

会拆（split）、会合（join）、会洗（strip），你就会处理大部分文本了 ✂